Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación

Práctica de Laboratorio Número 3

Profesor:
José Canache
Arquitectura del Computador

Autor:
Ronald Montilla
C.I. 32.011.453

2 de Octubre de 2026

Índice

1. La entrada/salida Mapeada en Memoria **2**
1.1. La estrategia del mapeo de memoria y su región en la memoria principal . 2
1.2. Implicaciones de las instrucciones lw y sw 2

2. Diferencias entre memory-mapped I/0 y port-mapped I/O **2**
2.1. Ventajas y Desventajas 3

3. La filosofía de MIPS32 y su influencia a la hora de utilizar memory-mapped I/O **4**

4. Problemas que surgen y sus soluciones **4**
4.1. Solapamiento de periféricos 4
4.2. ¿Acceso a memoria RAM o periférico? 4
4.3. Seguridad 5
4.4. Técnicas de esperas 5
4.4.1. Polling 5
4.4.2. Entrada/Salida por interrupciones 5
4.4.3. Acceso Directo a Memoria 6

5. Análisis y Discusión de los Resultados **6**

6. Bibliografía **7**

1. La entrada/salida Mapeada en Memoria

1.1. La estrategia del mapeo de memoria y su región en la memoria principal

La estrada/salida mapeada en memoria(memory-mapped I/O) es el método estándar en la actualidad que usan la mayoría de procesadores para comunicarse con los dispositivos de entrada/salida, esto funciona mediante el usó de una parte del espacio de direcciones para asociar las direcciones y registros de los periféricos de entrada/salida con el procesador. Es decir, se asignan direcciones de este espacio global directamente a los dispositivos periféricos.

Ahora bien, surge una pregunta muy importante sobre este método de comunicación entre el procesador y los dispositivos de estrada/salida, ¿En qué región de memoria se suelen mapear los dispositivos? La respuesta es que por convención se mapean las direcciones de la región más extrema del espacio de memoria, tanto para simplificar procesos del procesador como para proteger los datos de los dispositivos que están mapeados en las direcciones de está región, en resumen, la región en la que se mapean estas direcciones es la región más extrema(las ultimas) del espacio de memoria.

1.2. Implicaciones de las instrucciones lw y sw

¿Qué implicaciones tiene para las instrucciones lw y sw? Estas instrucciones son muy importantes y representan unas de las grandes ventajas del método memory-mapped I/O, y es que debemos saber que para métodos diferentes como el isolated I/O existen instrucciones específicas y exclusiva para interactuar con las direcciones de los periféricos, en la entrada/salida mapeada en la dirección esto no pasa debido a que las mismas instrucciones usadas para leer o escribir en la memoria principal pueden ser usadas para leer o escribir en las direcciones mapeadas de los periféricos, esto representa grandes ventajas de portabilidad que veremos más adelante.

2. Diferencias entre memory-mapped I/0 y port-mapped I/O

La diferencia entre entrada/salida mapeada en memoria(memory-mapped) y la entrada/salida por puertos(port-mapped) radica principalmente en el espacio de direcciones que emplean. Mientras un método(memory-mapped) usa una parte del espacio general de direcciones de la memoria, el otro(port-mapped) emplea un espacio de direcciones de E/S separado y no las direcciones del espacio de direcciones de la memoria, debido a que las direcciones viven en lugares diferentes todo cambia, para mapeo de memoria los datos viajan por los mismos buses, pero es un hardware externo a la CPU (el decodificador del bus) el que intercepta la dirección y sabe cuándo rutearla hacia un periférico y cuándo hacia la RAM. Pero con el mapeo de puerto es diferente porque el procesador requiere nuevas señales de control para indicar que una dirección pertenece a un periférico y cuando pertenece a la memoria principal. Solo con estas diferencias entre ambos métodos se producen muchas ventajas y desventajas para ambos métodos.

En el pasado los procesadores tenían arquitecturas con capacidades muy limitadas, tenían rangos de direcciones muy limitadas, por ello muy pocos procesadores empleaban el mapeo de memoria para los periféricos, pero gracias a los grandes avances en arquitectura de procesadores todo cambió y los procesadores de la actualidad emplean más este método por sus múltiples ventajas para algunas aplicaciones.

2.1. Ventajas y Desventajas

Memory-mapped I/O: Como ya sabemos, este método usa una parte del espacio de direcciones de la memoria para los dispositivos de entrada y salida, este es el método más usado por los procesadores modernos, y con esto nos podemos hacer una idea de las grandes ventajas que puede traer este método a los procesadores que la usen, pero así como tiene muchas ventajas, también tiene desventajas.

Ventajas:

- Mayor facilidad para interactuar con los periféricos ya que este método permite el usó de las mismas instrucciones que se emplean para manipular datos de la memoria para el manejo de los periféricos de E/S como lo son las instrucciones lw y sw.
- Mayor rapidez a la hora de acceder a los dispositivos de entrada y salida ya que se emplean los mismos mecanismos que para acceder a la memoria principal.

Desventajas:

- Reducción del espacio de direcciones disponible para la memoria principal, ya que un bloque del mapa lógico queda reservado exclusivamente para los periféricos.
- Menor capacidad de periféricos porque posee un espacio de direcciones compartido entre los dispositivos de entrada/salida y la memoria principal.

Port-mapped I/O: Los procesadores con port-mapped establecen puertos entre los dispositivos E/S, de está manera acceden a ellos. Pero requieren de señales de control e instrucciones especializadas y exclusivas.

Ventajas:

- Al usar espacio de direcciones aparte del espacio de direcciones de la memoria, este método no quita capacidad de almacenamiento a la memoria principal.
- Espacio de direcciones propio, lo que permite una mayor flexibilidad con la cantidad o manipulación de los periféricos.

Desventajas:

- Requieren instrucciones especializadas para acceder a estos periféricos lo que a sus sistemas menos portables y que los programados deban aprender y familiarizarse con nuevas instrucciones.
- Nuevas señales de control lo que hace a los procesadores más complejos.
- La programación se vuelve más compleja.

3. La filosofía de MIPS32 y su influencia a la hora de utilizar memory-mapped I/O

La filosofía de MIPS32 es RISC(Reduced Instruction Set Computer) que básicamente busca hacer todo lo más simple posible, esto se traduce en usar la menor cantidad de instrucciones en su arquitectura. La entrada/salida por puerto requiere usar espacios de direcciones separadas del espacio de direcciones de la memoria RAM, y para manipular datos de estos puertos no se emplean las mismas instrucciones que se emplean para manipular la memoria RAM. Se usan instrucciones específicas y exclusivas para esos puertos. Algunas de estas instrucciones son IN y OUT que permiten almacenar y leer datos. ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O? Utiliza principalmente la entrada/salida mapeada en memoria ya que encaja perfectamente con su filosofía al reutilizar instrucciones como lw y sw en vez de tener que utilizar más instrucciones como IN y OUT, aparte de emplear los mismos mecanismos que emplea a la hora de accerder o ingresar datos en la memoria RAM, también facilita la seguridad de los datos de los periféricos al mapear las direcciones de los perifericos en el extremo del campo de direcciones.

4. Problemas que surgen y sus soluciones

4.1. Solapamiento de periféricos

En las arquitecturas que emplean el mapeo de memoria pueden surgen varios problemas e inconvenientes, uno de ellos es el solapamiento que se produce cuando dos dispositivos periféricos emplean la misma dirección, cuando la CPU intenté comunicarse con estos periféricos puede hacerlo con el incorrecto, puede provocar un cortocircuito si ambos periféricos intentan comunicarse al mismo tiempo con la CPU y también puede leer o almacenar datos incorrectos, pero esto es un caso hipotético y tiene sus soluciones. Las soluciones para este primer caso son muy inteligentes, la placa base tiene circuitos lógicos que toman la dirección del bus y la procesan para indicar mediante un pin(Chip Select) el dispositivo exacto a usar, de tal manera que uno se activa y el otro se mantiene desactivado evitando acceder a dispositivos y datos incorrectos. Otra cosa que no permite que esto suceda es la asignación dinámica de direcciones. Cuando se arranca un PC, el sistema operativo o la BIOS interactúan con cada dispositivo de entrada/salida para verificar su consumo exacto de direcciones y asignarle dinámicamente rangos de direcciones específicas a cada dispositivo, de tal manera que cada dispositivo tenga su propio rango de direcciones.

4.2. ¿Acceso a memoria RAM o periférico?

Otra cuestión es la siguiente, ¿Como sabe el hardware si debe acceder a un dispositivo periférico o a la memoria RAM? Claro, si la CPU envía una dirección por el bus para obtener su dato, ¿Como el hardware puede determinar si esa dirección pertenece a una variable en Memoria RAM o a un dato de un dispositivo de entrada/salida? Este trabajo lo hace una parte de hardware externo al procesador, lo hace el *decodificador de direcciones*, cuando el procesador envía una dirección para leer su dato, este no sabe si ese dato viene de la memoria RAM o de un dispositivo periférico porque su trabajo consiste en enviar la señal, recibir el dato y procesarlo. Pero cuando envía está dirección por el bus de direcciones, el decodificador de direcciones la toma y lee sus bits más significativos con la tarea de determinar si esa dirección pertenece a la región de direcciones más externa, si

pertenece a esa región significa que la dirección pertenece a un periférico y mediante un pin(Chip Select) manda una señal para deshabilitar la memoria RAM y obtener el dato del lugar correcto, es decir, del dispositivo de entrada/salida. Si no pertenece a la región más externa significa que el dato que la CPU pidió, proviene de la memoria RAM. En resumen, es el *decodificador de direcciones*, el que determina si se debe leer o guardar un dato en la memoria principal o en un dispositivo de entrada/salida.

4.3. Seguridad

En cuanto a la seguridad de los datos, ¿es posible que un programa sin privilegios acceda a un periférico mapeado en memoria? La respuesta a esto es que en la actualidad es casi imposible, gracias al estricto trabajo conjunto entre hardware y software.

En primer lugar, los sistemas operativos asignan a cada programa bloques de memoria virtual. Cuando un programa interactúa con una de estas direcciones virtuales, una unidad de hardware integrada en el procesador (*La Unidad de Manejo de Memoria - MMU*) la traduce en tiempo real a la dirección física correspondiente. De está manera, un programa nunca interactúa con las direcciones físicas reales del hardware, sino que lo hace con direcciones "falsas.º aisladas que le asigna el sistema operativo. Aparte de la virtualización, al momento de crear estas tablas de direcciones, el sistema operativo implementa niveles de seguridad. A los programas comunes se les asigna un *modo usuario*, el cual tiene permisos sumamente restringidos. Por el contrario, el *modo kernel* (que poseen el procesador y el núcleo del sistema operativo) tiene acceso absoluto y total libertad sobre el hardware y los diferentes dispositivos de E/S. El sistema operativo marca las direcciones físicas de los periféricos para que exijan privilegios de modo kernel. Es por todo lo antes mencionado que, si un programa en modo usuario intenta adivinar y acceder a los datos de un periférico o actuar de forma maliciosa sobre estos, la MMU detectará la falta de privilegios y abortará la instrucción. El hardware bloqueará el acceso al bus y notificará inmediatamente al sistema operativo para que detenga o cierre el programa malicioso.

4.4. Técnicas de esperas

4.4.1. Polling

Ante un procesador que solicita un dato pero aún no lo tiene disponible surge una de las principales técnicas de espera, *la espera activa(polling)* que consiste en atrapar al procesador en un bucle hasta que obtenga el dato, esto estaba bien para procesadores que solo realizan una tarea, pero para procesadores que realizan más de una, incluso millones de tareas, está técnica no es la ideal por el tiempo que puede tomar esperar este dato y porque lógicamente no es eficiente. Para solucionar este problema surgen nuevas técnicas que hoy en día son empleadas por los procesadores moderno.

4.4.2. Entrada/Salida por interrupciones

En la actualidad los procesadores realizan miles de tareas al mismo tiempo y no pueden perder eficiencia y ciclos de reloj esperando datos, es por ello que los procesadores usan técnicas de interrupciones. Cuando un procesador requiere de un dato de un periférico pero no lo tiene disponible aún, este no lo espera sino que hace las demás tareas hasta que el periférico tiene el dato disponible para ser llevado ante el procesador y mediante una *Línea de Interrupción* el periférico envía una señal al procesador interrumpiendo las

tareas que estaba haciendo para que tome el dato y lo procese, luego de procesar el dato volverá a donde estaba.

4.4.3. Acceso Directo a Memoria

Las interrupciones a la CPU son buenas en la mayoría de los casos, pero cuando se producen demasiadas de estas la eficiencia se pierde, ante esto surge como solución el *Acceso Directo a Memoria(DMA)*, un componente integrado en la placa base que permite mover datos de un componente a otro sin la intervención a cada momento de la CPU. Esto permite que antes el problema de movilizar y esperar grandes cantidades de datos de los periféricos de E/S el DMA se encargue mientras la CPU realiza otras tareas, potenciando la eficiencia a la hora de esperar y mover datos desde los periféricos a la RAM.

5. Análisis y Discusión de los Resultados

En los archivos adjuntos(sensorLuminosidad`.asm` y tensionArterial`.asm`) se implementó en lenguaje ensamblador MIPS32 el controlador para un sensor de luminosidad y un módulo para medir la tensión arterial, y evidenció lo siguiente:

Se pudo observar la influencia de la filosofía RISC y el usó del mapeo en la memoria al usar las instrucciones *sw* y *lw* para acceder o guardar datos en los registros de los periféricos pero que originalmente se usan para manipular datos de la memoria RAM. Gracias a esto, la implementación de estos periféricos fue más fácil al no tener que manipular instrucciones específicas para los datos de los periféricos.

En ambas implementaciones se usó la espera activa(polling), en el sensor de luminosidad la espera mantuvo en espera al procesador hasta que el registro mapeado en memoria(luzEstado) cambiara su valor de 0 a 1 o -1 para luego continuar con la demás lógica. En el módulo para la tensión arterial también se empleo la espera activa para mantener al procesador en un bucle mientras el registro mapeado en memoria(tensionEstado) no cambie de 0(midiendo) a 1(resultados listos). La espera activa en estos dispositivos es correcta pero también se comprobó que al ser una simulación se debían cambiar los datos de los registros manualmente, en MARS esto se logra porque es un entorno educativo, pero si realmente el usuario accediera a un registro de un dispositivo periférico mapeado en memoria la Unidad de Manejo de Memoria bloquearía este intento al no poseer los permisos necesarios o interpretarlo como algo ilegal.

En resumen, fue posible constatar la implementación de los controladores de los periféricos correctamente, pero debido a que MARS es un entorno educativo, se observaron limitaciones al tener que interactuar directamente con los datos, pero se logró observar las diferentes implicaciones del mapeo de memoria.

6. Bibliografía

- Anfieo. 2025/07/03. ¿Qué diferencia hay entre la E/S mapeada en memoria y la E/S mapeada en puertos?. Medium. https://www.geeksforgeeks.org/computer-organization-arc memory-mapped-i-o-and-isolated-i-o/
- GeeksForGeeks. 2025/11/07. E/S mapeadas en memoria y E/S aisladas. GeeksForGeeks. https://www.geeksforgeeks.org/computer-organization-architecture/memory-mapped-i-o-and-isolated-i-o/
- David A. Patterson, John L. Hennessy (2011). Estructura y diseño de computadores. Editorial Reverté, S. A.
- E/S mapeada en memoria. (2026, 2 de octubre). En *Wikipedia*. https://es.wikipedia.org/wiki/E/S_mapeada_en_memoria